

Algorithmique parallèle || Fiche :)

1 Évaluation des Performances et Lois Fondamentales

1.1 Variables de Base

Avant tout calcul, il faut définir ton environnement :

- n : Taille du problème (ex: nombre de pixels, taille d'un tableau à trier).
- p : Nombre d'unités de calcul physiquement disponibles (threads, cœurs, machines).
- $T_1(n)$: **Temps séquentiel** (temps mis avec une seule unité, l'algorithme de base).
- $T_{p(n)}$: **Temps parallèle** (attention : ce temps s'arrête uniquement quand la **dernière** unité a terminé son travail).
- T_∞ : Temps parallèle idéal, dans l'hypothèse où l'on disposerait d'un nombre infini d'unités.

1.2 Degré de Parallélisme et Coût Élémentaire

Ces notions permettent de construire théoriquement T_1 et T_∞ étape par étape.

- **Degré** : Nombre maximum d'opérations indépendantes faisables **simultanément** à une étape donnée.
- **Coût élémentaire** (t_c) : Temps d'exécution requis par **une seule** machine pour faire **une seule** de ces opérations.
- **Formules** :
 - $T_1 = \sum (\text{Degré} \times \text{Coût élémentaire})$;
 - $T_\infty = \sum \text{Coût élémentaire (degré absorbé par le parallélisme infini)}$.

1.3 Les 3 Métriques Essentielles

Une fois les temps T_1 et T_p connus, on évalue la qualité de l'algorithme :

- **Accélération (Speedup)** : $S_p = \frac{T_1}{T_p}$. *Signification* : Combien de fois l'algorithme va plus vite. Max théorique = p .
- **Travail (Work)** : $W_p = p \times T_p$. *Signification* : Quantité totale de ressources machines mobilisées (et payées !).
- **Efficacité (Efficiency)** : $E_p = \frac{S_p}{p} = \frac{T_1}{W_p}$. *Signification* : Ratio des ressources **utilement** exploitées, compris entre 0 et 1 (100%). Plus on ajoute des machines (p), plus l'efficacité a tendance à baisser.

1.4 Loi d'Amdahl et Temps Parallèle (T_p)

La loi d'Amdahl modélise le fait qu'un algorithme n'est jamais 100% parallèle. Il possède une fraction strictement séquentielle R_s et une fraction parallélisable $(1 - R_s)$.

- **Calcul du Temps Parallèle via Amdahl** : Seule la portion parallèle est divisée par p .

$$T_p = T_1 \times \left(R_s + \frac{1 - R_s}{p} \right)$$

Astuce d'examen : Si un sujet donne uniquement des pourcentages (ex: $R_s = 70\%$), fixe simplement $T_1 = 1$. Le temps T_p obtenu sera directement utilisable en ratio.

- **Calcul de l'Accélération (Speedup)** :

$$S_p = \frac{1}{R_s + \frac{1 - R_s}{p}}$$

- **La Limite Théorique** : Si on a une infinité de processeurs ($p \rightarrow \infty$), l'accélération plafonne. Il est impossible d'aller plus vite que :

$$S_\infty = \frac{1}{R_s}$$

- **Impact du Degré sur Amdahl** : Si la partie parallélisable a un degré maximum d (ex: $d = 6$), il est inutile d'utiliser un nombre d'unités $p > d$. On fixe alors obligatoirement $p = d$ dans la formule pour trouver l'accélération maximale.

1.5 Le Principe de Brent (Optimisation de l'Efficacité)

Ce principe stipule qu'on peut simuler un algorithme conçu pour une infinité d'unités sur un système avec p unités restreintes, et remonter ainsi à 100% d'efficacité.

- **Formule prédictive** : $T_p = O\left(\frac{T_1}{p} + T_\infty\right)$.
- **Exemple classique (La Réduction d'un tableau)** : Additionner n nombres en arbre.
 - Avec $p = n$ unités : Le temps est très bon $T_n = \log_2(n)$, mais l'efficacité est catastrophique $E_n = \frac{1}{\log_2}(n)$ (beaucoup dorment à la fin).

- *Application de Brent* : On réduit sciemment le nombre d'unités à $p = \frac{n}{\log_2(n)}$. Le temps d'exécution reste proportionnel à $O(\log_2(n))$, mais **l'efficacité remonte à 1**.

1.6 Performances du Parallélisme de Pipeline

Utilisé quand on découpe le traitement d'un flux de données en k étapes successives de durée égale t .

- **Temps total** pour traiter n données : $T_k = k \times t + (n - 1) \times t$.
- **Interprétation** : Le pipeline **ne réduit pas** le temps de calcul pour une donnée individuelle, il augmente le **débit global**.
- **Goulot d'étranglement** : Si les étapes n'ont pas la même durée, le rythme est dicté par l'étape la plus lente. La solution est de dupliquer physiquement cette étape.

1.7 7. Rédaction du Pseudo-Code Parallèle

Règles de base et Sécurité Mémoire : Utiliser pour u allant de 0 à $P-1$ faire en parallèle. Toujours créer des variables locales pour chaque unité (ex: indexer $i[u]$) pour éviter le non-déterminisme.

Exemple :

```
pour u allant de 0 à P-1 faire en parallèle :
  i[u] <- u
  tant que i[u] < n faire :
    C[i[u]] <- A[i[u]] + B[i[u]]
    i[u] <- i[u] + P
  fintantque
finpour
```

Exemple 2 : Répartition par Blocs

```
reste <- n % P
pour u allant de 0 à P-1 faire en parallèle :
  nbCases[u] <- floor(n/P)
  si u < reste alors
    nbCases[u] <- nbCases[u] + 1
    deb[u] <- u * nbCases[u]
  sinon
    deb[u] <- u * nbCases[u] + reste
  finsi

  // La boucle de calcul elle-même est strictement SÉQUENTIELLE
  pour i[u] de deb[u] à deb[u] + nbCases[u] - 1 faire :
    C[i[u]] <- A[i[u]] + B[i[u]]
  finpour
finpour
```

Utilisation des opérateurs formels (α et β) :

- $\alpha(op, v_1, v_2, n)$: Applique un opérateur op élément par élément.
- $\beta(op, v, n)$: Réduit le vecteur v en une valeur (Ex: somme d'une ligne l avec $\text{beta}(+, \text{ligne}(\text{tab}, l), n)$).